

课程笔记

LLM, Agents 和 Scaling Law

南京大学 操作系统原理 2026 春季学期 第 04 讲 (Aside)

lecture-to-notes

2026 年 3 月 24 日



视频作者/频道: 绿导师原谅你了 (蒋炎岩 jyy)

发布日期: 2026 年春季学期

视频时长: 1 小时 22 分钟

视频链接: <https://www.bilibili.com/video/BV1gEcmzzE4P>

目录

1 大 & 语言 & 模型	2
1.1 大 (Large)	2
1.2 语言 (Language)	2
1.3 模型 (Model)	2
1.4 预训练与对齐	3
1.5 本章小结	3
2 Agents	4
2.1 Agent 能力的进化	4
2.2 MCP: Model Context Protocol	5
2.3 本章小结	5
3 Scaling Law	6
3.1 Elo 评分系统	6
3.2 Scaling Law: 算力 $\rightarrow$ 能力	6
3.3 推理时 Scaling	7
3.4 本章小结	7
4 豆包帮我逃课操作系统实验	8
4.1 软件工程中的复杂性分解	8
4.2 豆包实战: Vibe Coding	9
4.3 本章小结	9
5 总结与延伸	9
5.1 讲者的核心信息	9
5.2 结构化提炼	10
5.3 跨章节关联	10
5.4 与前三讲的关联	10
5.5 开放问题	10
5.6 拓展阅读	11

1 大 & 语言 & 模型

这是操作系统课程的一节“Aside”——jyy 暂停操作系统主线，用一整节课讲清楚大语言模型的核心原理。为什么？因为在 Agentic AI 时代，**理解 LLM 是理解操作系统未来的前提**。

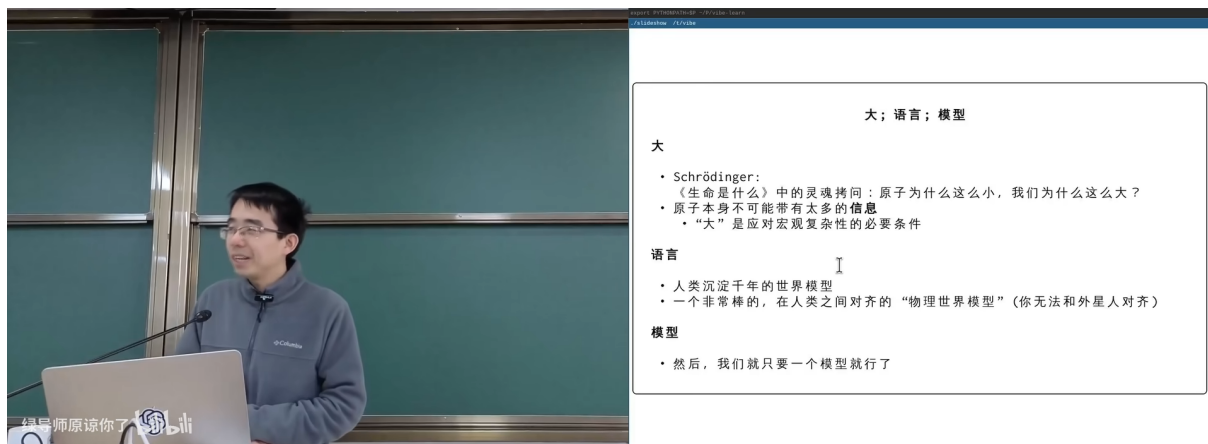


图 1: 拆解“大语言模型”的三个关键词¹

jyy 将“大语言模型”拆解为三个独立的概念：

1.1 大 (Large)

引用 Schrödinger 的《生命是什么》：原子为什么这么小？我们为什么这么大？

“大”的本质

- 原子本身不可预测，但足够大的物体行为就可以预测
- “大”是因果规律的涌现的必要条件
- 类比：足够多的参数 → 涌现出“理解”的能力

1.2 语言 (Language)

- 人类仅凭千年的世界模型
- 一个非常粗糙的、在人类之间对齐的“地理世界模型”（你无法和外星人对齐）

1.3 模型 (Model)

“然而，我们真正需要的只是一个模型就行了。”——关键洞察：LLM 不需要理解世界的全部，只需要建立一个**足够好的近似模型**。

¹ 视频画面时间区间：00:04:45–00:05:30。

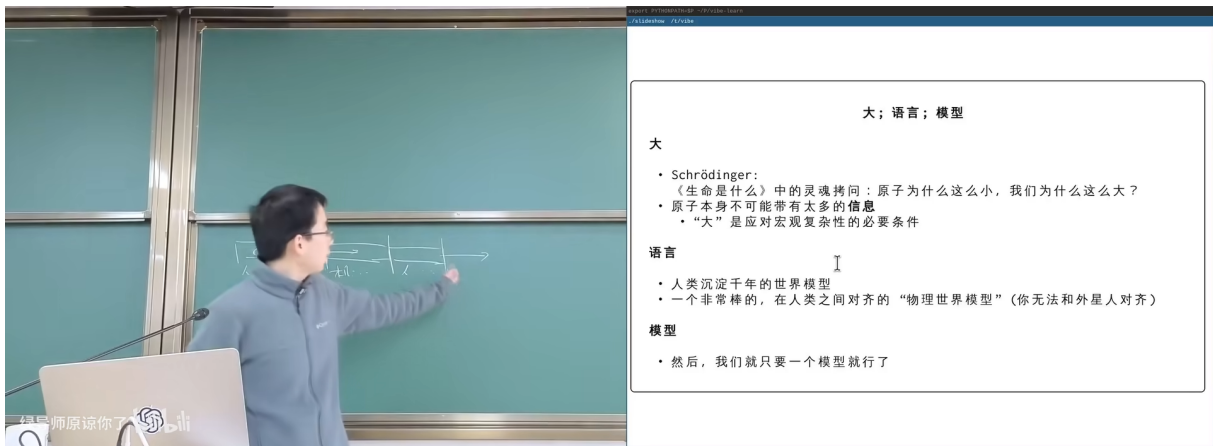


图 2: Transformer 架构与注意力机制²

1.4 预训练与对齐

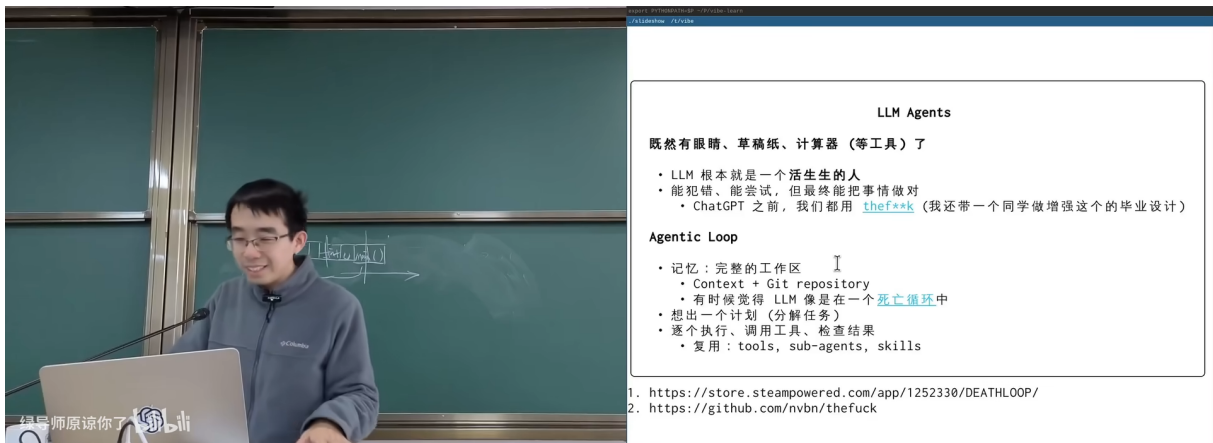


图 3: 预训练数据与模型能力的关系³

从预训练到有用的 AI

预训练 (Pre-training): 在海量文本上学习“预测下一个 token”。模型在这个过程中内化了语言结构、世界知识和推理模式。

对齐 (Alignment / RLHF): 让模型从“会说话”变成“会帮忙”。没有对齐的 LLM 像一个读了所有书但不懂社交的人。

涌现 (Emergence): 当模型足够大时，出现了训练目标中没有显式要求的能力（如数学推理、代码生成）。这就是“大”的力量。

1.5 本章小结

大语言模型 = 大（参数量带来涌现）+ 语言（人类共享的世界模型接口）+ 模型（近似而非精确的世界表示）。理解这三个维度，是理解后续 Agent 和 Scaling Law 的基础。

²视频画面时间区间：00:07:00–00:07:45。

³视频画面时间区间：00:14:00–00:14:45。

2 Agents

从“模型”到“代理”——LLM 如何从被动回答问题变成主动解决问题？

2.1 Agent 能力的进化

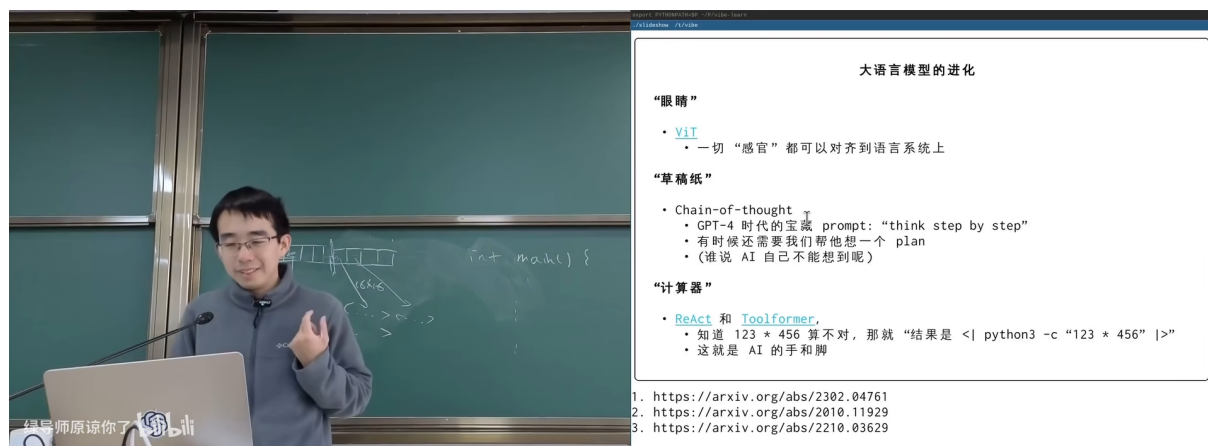


图 4: 大语言模型的进化：从感知到计算器到 React⁴

jyy 梳理了 Agent 能力的进化路径：

“感知” (XLS)

- 一回“感官”都可以对齐到语言系统上

“思维链” (Chain-of-Thought)

- GPT-4 时代的宝藏 prompt: “think step by step”
- 相当于这是在帮我们想出一个 plan
- (虽然 AI 自己不能确保自己想的对)

“计算器” (React & Toolformer)

- 知道 $123 + 456$ 算不对，那就“结果是 `| python3 -c "123 + 456" |`”
- 这就是 AI 的外挂

Agent 的核心：感知 + 推理 + 工具

Agent 不是一个更聪明的 chatbot。它是 LLM + 工具调用 + 规划能力的组合。关键进化：

1. **感知**：多模态输入（文本、图像、音频）
2. **推理**：Chain-of-Thought 让 LLM “先想再做”
3. **工具**：React/Toolformer 让 LLM 调用外部工具（计算器、搜索引擎、代码执行器）
4. **行动**：MCP 等协议让 Agent 能操作真实系统

⁴视频画面时间区间：00:24:30-00:25:15。

2.2 MCP: Model Context Protocol

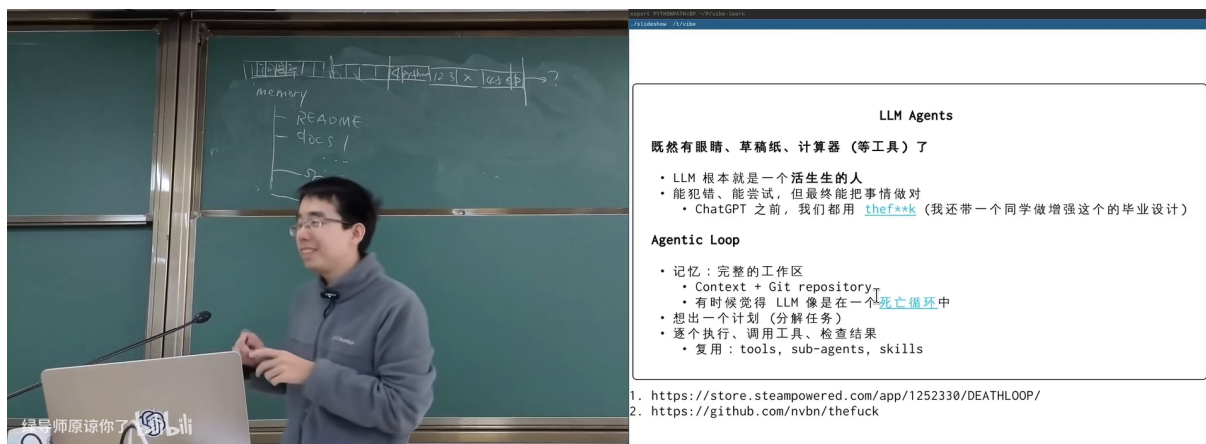


图 5: MCP 协议与 Agent 的工具生态⁵

MCP 对操作系统的意义

MCP (Model Context Protocol) 是 Agent 与外部工具交互的标准协议。类比操作系统的 syscall——

- syscall 是程序与 OS 的接口
- MCP 是 Agent 与工具生态的接口

jyy 的洞察：MCP 之于 Agent，就像 POSIX 之于 UNIX——它定义了一个标准化的工具调用接口，使得任何 Agent 都可以使用任何工具。

2.3 本章小结

Agent = LLM + 感知 + 推理 (CoT) + 工具 (React) + 协议 (MCP)。这不是增量改进——而是从“被动回答”到“主动行动”的质变。MCP 之于 Agent 就像 syscall 之于程序。

⁵ 视频画面时间区间：00:30:00–00:30:45。

3 Scaling Law

3.1 Elo 评分系统

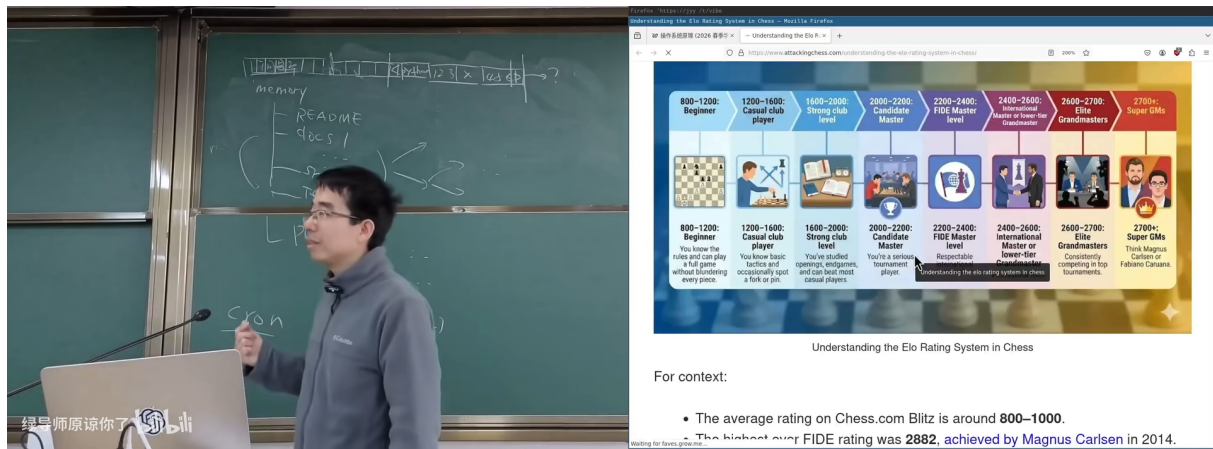


图 6: 国际象棋 Elo 评分系统：从业余到超级大师⁶

jyy 用国际象棋的 Elo 评分系统来解释 AI 的能力度量：

- 业余水平：800 – 1000
- 普通玩家：1000 – 1600
- 高手：1600 – 2200
- 大师：2200 – 2500
- 超级大师：2500+
- Magnus Carlsen (2014 最高纪录)：2882

关键点：Chess.com 平均 rating 约 800 – 1000。

3.2 Scaling Law: 算力 → 能力

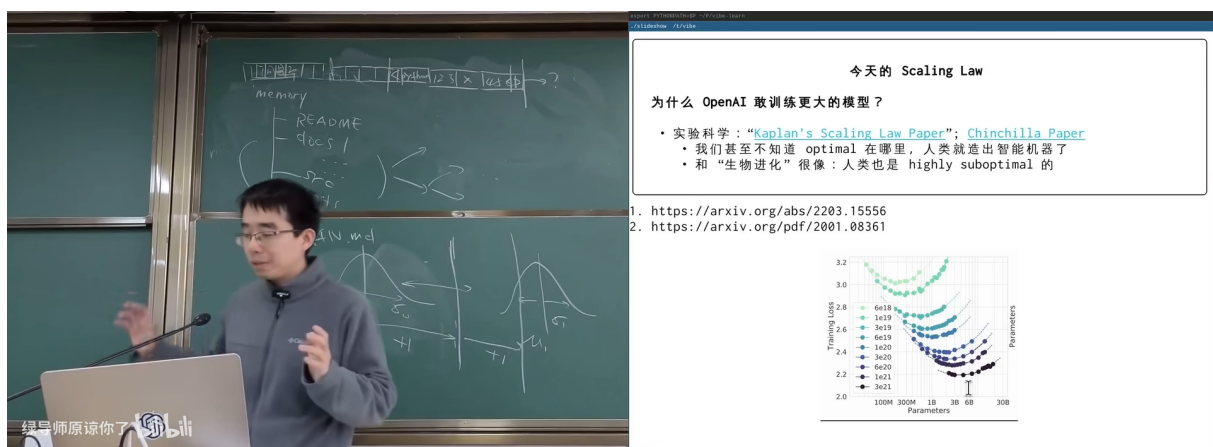


图 7: Scaling Law: 模型大小与能力的关系⁷

⁶视频画面时间区间：00:44:30–00:45:15。

Scaling Law 的核心发现

OpenAI 的 Scaling Law (Kaplan et al., 2020) 发现：
模型的 loss 与三个因素呈幂律关系 (power law)：

$$L(N, D, C) \propto N^{-\alpha_N} + D^{-\alpha_D} + C^{-\alpha_C}$$

- N ：模型参数量
- D ：训练数据量
- C ：计算量 (FLOPs)
- α ：缩放指数 (经验常数)

含义：只要持续增加算力、数据和参数，模型能力就会持续提升——而且是可预测的提升。

3.3 推理时 Scaling

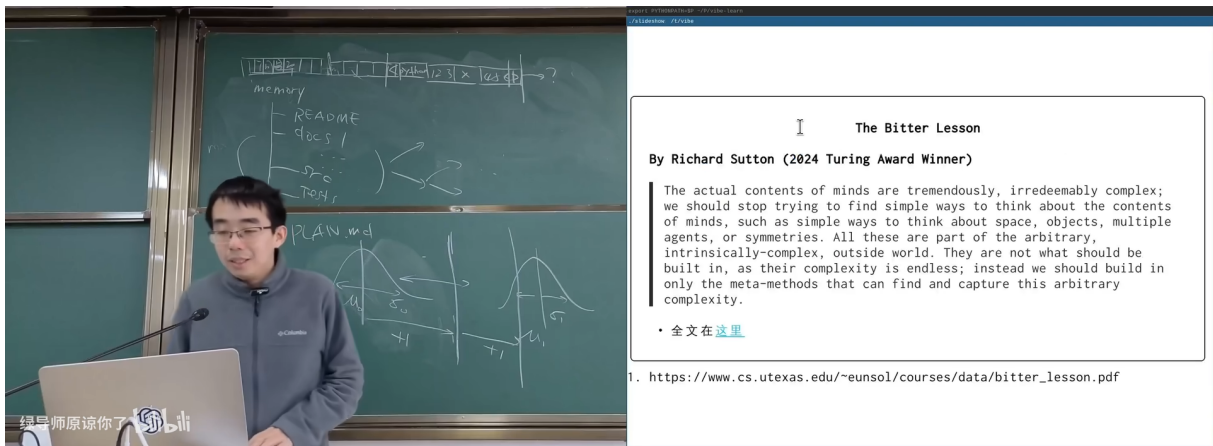


图 8: 推理时 Scaling：用更多推理计算换取更好的结果⁸

训练时 Scaling vs. 推理时 Scaling

训练时 Scaling (2020-2023)：更大的模型 + 更多数据 = 更好的基础能力。代表：GPT-3 → GPT-4。

推理时 Scaling (2024+)：同一个模型，用更多的推理计算（如 Chain-of-Thought、Tree Search、多次采样）= 更好的回答。代表：o1、deepseek-r1。

类比：训练时 Scaling 像“上更多年的学”；推理时 Scaling 像“考试时想得更久”。两者正交，可以叠加。

3.4 本章小结

Scaling Law 是 AI 时代最重要的经验发现之一：能力与算力/数据/参数的关系是可预测的幂律。推理时 Scaling 开辟了第二条提升路径。这解释了为什么 AI 进步如此之快——不是偶然突破，

⁷ 视频画面时间区间：00:48:30-00:49:15。

⁸ 视频画面时间区间：00:53:30-00:54:15。

而是系统性的资源投入。

4 豆包帮我逃课操作系统实验

这是本讲最有趣的章节——jyy 让 AI（豆包/字节跳动的模型）帮忙做操作系统实验，现场展示 AI 编程的能力和局限。

4.1 软件工程中的复杂性分解

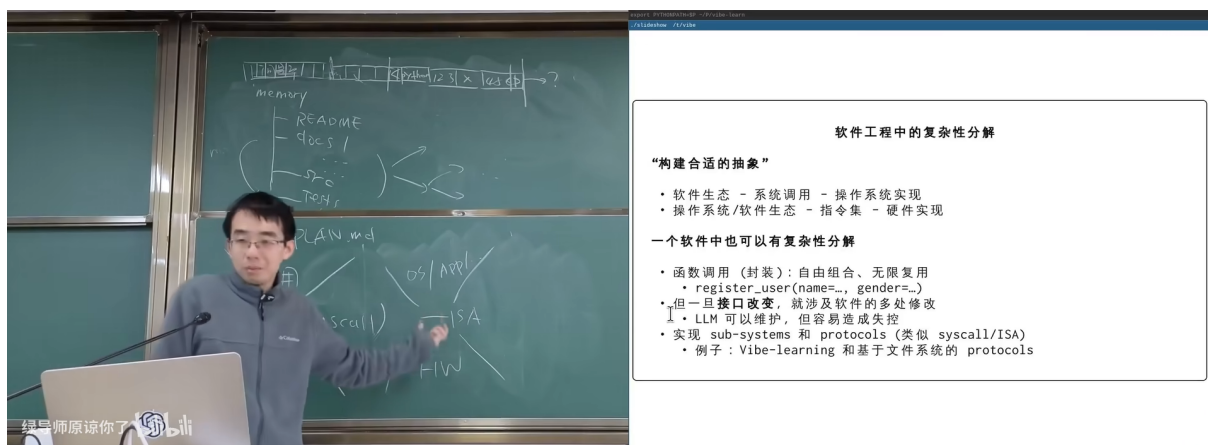


图 9: 软件工程中的复杂性分解：构建合适的抽象⁹

jyy 从软件工程的角度分析了 AI 编程的本质：

“构建合适的抽象”

- 软件生态 → 系统调用 → 系统实现
- 操作系统/软件生态 → 接口集 → 硬件集成
- 一个软件中也可以有复杂性分解：
 - 函数接口（解耦）：自由组合，互不影响
 - 例：`register_user(name, gender, ...)`
 - 进一步接口文档，检测方法已经变成了“生存的刚需”
 - 上层 LLM 的对话就变成了一种接口实现
- 实现 sub-systems 和 protocols（类似 `syscall/ISA`）
- 类似于 Vibe-learning 框架于文件系统的关系

⁹视频画面时间区间：01:03:30–01:04:15。

4.2 豆包实战：Vibe Coding

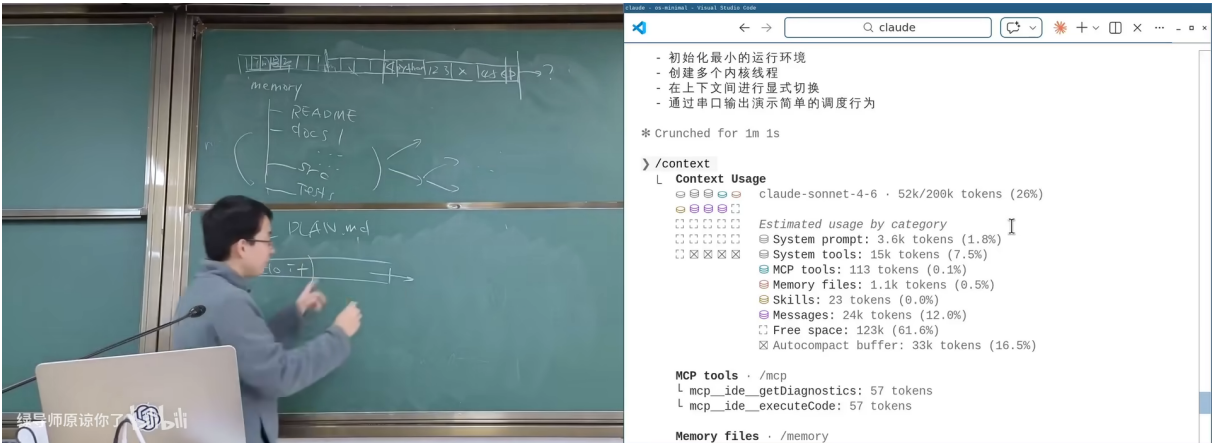


图 10: 豆包/AI 辅助编程的实战演示¹⁰

jyy 现场让豆包帮忙完成操作系统实验。关键观察：

- AI 可以快速生成框架代码和常见模式
- 但在**系统级编程**中，AI 经常犯细节错误（寄存器名、ABI 约定、并发安全）
- “Vibe Coding”（凭感觉编程）在应用层可以工作，但在操作系统层危险重重

Vibe Coding 的边界

jyy 的警告：AI 辅助编程（“Vibe Coding”）有明确的适用边界——

适合：高层应用逻辑、CRUD、UI 布局、脚本自动化

不适合：操作系统内核、并发原语、中断处理、内存管理

原因：这些底层代码的正确性依赖于**精确的硬件行为理解**，而当前 AI 的“世界模型”在硬件细节上不够精确。这恰恰是为什么学操作系统仍然重要。

4.3 本章小结

AI 可以帮你写代码，但不能帮你理解系统。“豆包逃课”的实验证明：AI 在高层抽象上有效，在底层细节上不可靠。操作系统课教你的正是 AI 不擅长的部分——硬件行为的精确理解和系统级抽象的设计。

5 总结与延伸

5.1 讲者的核心信息

这节 Aside 讲座在操作系统课程中加入了 AI 视角，核心信息：

1. LLM 的能力来自“大”——参数量带来涌现
2. Agent 是 LLM + 工具 + 规划——MCP 是新时代的“syscall”

¹⁰视频画面时间区间：01:11:30–01:12:15。

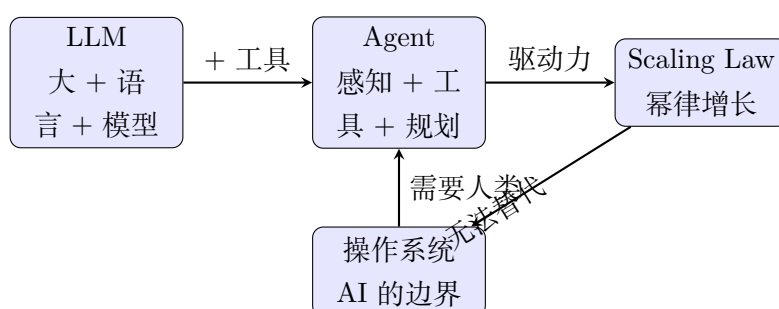
3. Scaling Law 保证了 AI 进步的可预测性
4. 但 AI 在系统级编程中仍不可靠——学操作系统仍然必要

5.2 结构化提炼

四个核心 Takeaway

1. **大 → 涌现**：足够多的参数让 LLM 展现出训练目标未显式要求的能力。这不是魔法——是统计物理的涌现现象在人工神经网络上的复现。
2. **Agent = LLM + 工具**：感知（多模态）→ 推理（CoT）→ 工具（React/Toolformer）→ 协议（MCP）。MCP 之于 Agent 就像 POSIX 之于 UNIX。
3. **Scaling Law 是可预测的**： $L \propto N^{-\alpha}$ 。AI 的进步不是偶然——而是资源投入的可预测回报。推理时 Scaling 是第二条增长轴。
4. **Vibe Coding 有边界**：AI 在应用层有效，在操作系统层不可靠。理解硬件行为和设计系统抽象——这是人类的不可替代能力。

5.3 跨章节关联



5.4 与前三讲的关联

- 第 01 讲说 “Code is cheap, show me the talk” → 第 04 讲解释了为什么：AI (LLM+Agent) 让代码变便宜，但 “talk” (系统设计能力) 变得更贵
- 第 02 讲介绍了 syscall 作为程序与 OS 的接口 → 第 04 讲类比 MCP 是 Agent 与工具的接口
- 第 03 讲说硬件不知道有没有 OS → 第 04 讲说 AI 也不真正 “理解” 硬件——Vibe Coding 在底层不可靠

5.5 开放问题

1. 当 Scaling Law 继续推进，AI 最终能否理解硬件级细节？
2. MCP 会成为 Agent 时代的 POSIX 吗？
3. 操作系统课程应该如何适应 AI 时代——教更多 “为什么”，还是教更多 “怎么用 AI”？

5.6 拓展阅读

- Kaplan et al. “Scaling Laws for Neural Language Models” (2020)
- Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models” (2023)
- Anthropic MCP 规范: 搜索 “Model Context Protocol specification”